Auditing Model Substitution in LLM APIs

Why software tests fail and hardware attestation wins (arxiv.org/abs/2504.04715)

LLM APIs are black boxes: users cannot verify the served model.

Economic incentives motivate covert swaps to cut GPU cost.

We review evidence that software-only auditing is unreliable, and why TEEs are actionable.

The Model Substitution Problem: Economic Incentives Drive Cov

Substitution types (H1)

(1) Smaller model (e.g., 8B instead of 70B)
(2) Quantized variant (FP8/INT8 instead of FP16)
(3) Fine-tuned / updated version
(4) Different model family

Hypothesis testing framing

H0: samples ~ P_spec(y|x)
H1: samples ~ P_actual(y|x)
Goal: distinguish distributions from API outputs
Constraint: realistic query budgets + deployment noise

Why this happens

Provider pricing is tied to ‘model name’, while costs scale with compute.
Small accuracy drops + black-box access create a profitable trust gap.
Audits must withstand subtle changes and adaptive adversaries.

Threat model: covert substitution behind an API

FP8 Quantization Preserves Most Accuracy — Making Substitutio

Original vs FP8-quantized scores (mean ± error) across four benchmarks

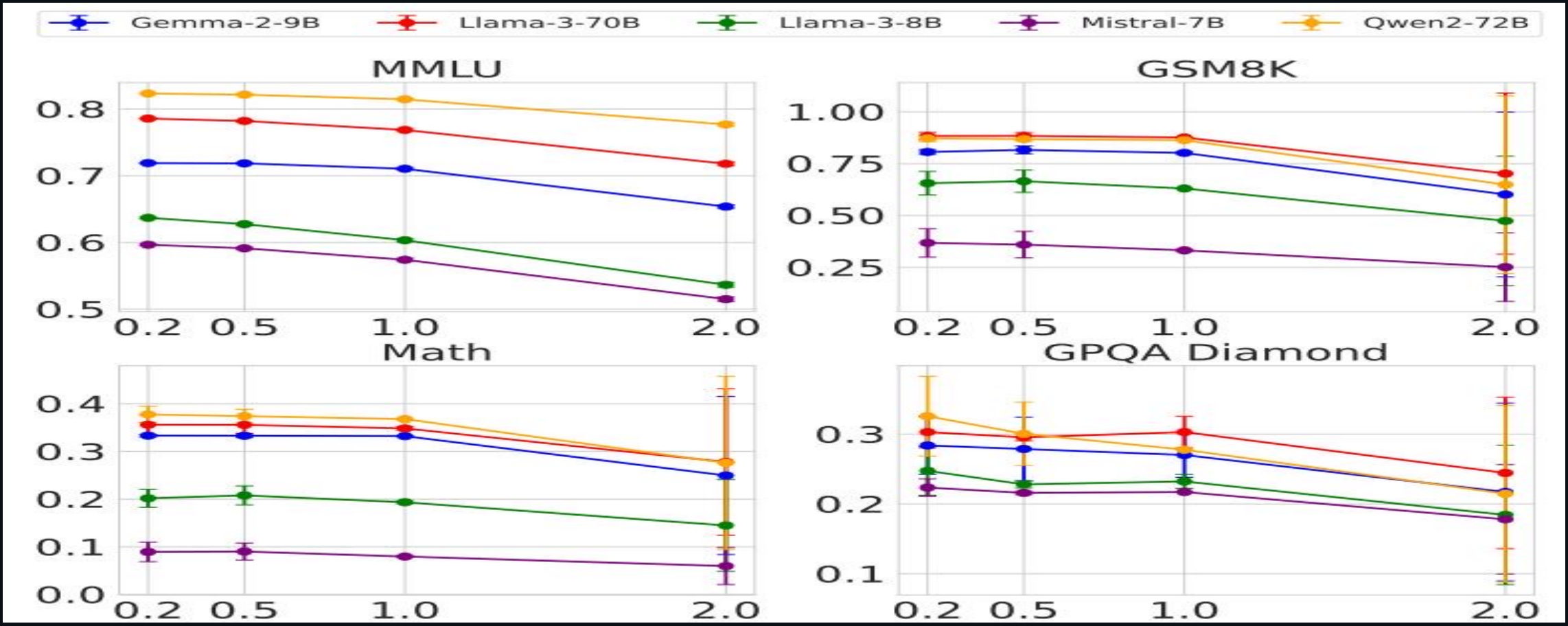
Model family	MMLU	GSM8K	MATH	GPQA Diamond
Llama-3-8B (orig / FP8)	62.69 ± 0.18 / 62.43 ± 0.26	61.14 ± 3.47 / 60.90 ± 4.10	20.65 ± 3.43 / 14.91 ± 2.49	22.62 ± 0.22 / 20.14 ± 0.24
Llama-3-70B (orig / FP8)	78.05 ± 0.08 / 77.88 ± 0.13	88.06 ± 1.44 / 87.35 ± 1.24	35.69 ± 1.33 / 35.75 ± 1.16	29.60 ± 0.51 / 33.12 ± 0.30
Gemma-2-9B (orig / FP8)	71.86 ± 0.08 / 71.92 ± 0.11	81.80 ± 1.35 / 79.41 ± 1.14	33.41 ± 0.28 / 32.53 ± 0.34	28.64 ± 2.97 / 27.81 ± 3.22
Qwen2-72B (orig / FP8)	82.18 ± 0.08 / 81.98 ± 0.08	86.72 ± 1.00 / 86.82 ± 0.97	37.39 ± 1.41 / 37.67 ± 1.39	29.93 ± 2.71 / 31.08 ± 1.96
Mistral-7B v0.3 (orig / FP8)	59.15 ± 0.10 / 58.77 ± 0.13	35.90 ± 4.54 / 32.20 ± 4.02	8.94 ± 1.24 / 7.68 ± 1.23	21.60 ± 0.17 / 22.72 ± 0.19

Implication

For many cells, FP8 deltas are within error bars → software detection needs large samples and careful controls.

Data: source_brief.md (arxiv 2504.04715)

Higher Temperature Amplifies Performance Gaps but Increases N



Observation

As temperature increases (0.2→2.0), accuracy degrades across models; the substitution signal narrows or becomes noisy.

Figure: benchmark/assets/figure_1.jpg

Log Probability Fingerprinting Fails Due to Inference Nondeterm

Model	BERT Acc	T5 Acc	GPT2 Acc	LLM2Vec Acc
Llama3-70B-Instruct-FP8	50.55	50.10	50.45	49.90
Llama3-70B-Instruct-INT8	51.60	49.90	51.30	50.25
Gemma2-9b-it-FP8	49.95	50.30	51.20	49.80
Gemma2-9b-it-INT8	49.00	49.70	51.65	49.55
Mistral-7b-v3-Instruct-FP8	50.55	49.75	48.70	48.75
Mistral-7b-v3-Instruct-INT8	49.70	50.75	50.50	51.15
Qwen2-72B-Instruct-FP8	50.05	50.25	48.75	49.80
Qwen2-72B-Instruct-INT8	50.75	50.55	49.45	50.20

Why this breaks audits

Same model can yield different token logprobs across software versions (transformers/vLLM) and GPUs (A100/H100).

An auditor cannot distinguish infra drift from substitution using logprobs alone.

Figure: benchmark/assets/figure_5.jpg

Randomized Routing at Low Rates Makes Statistical Detection Q

Attack: randomized substitution

Provider routes each query to the substitute with probability p.
Observed distribution: $P_{mix} = (1-p)\cdot P_{spec} + p\cdot P_{sub}$.

Consequence

As $p \to 0$, $P_{mix} \to P_{spec}$: any text-based test needs dramatically more samples.
Even strong tests become impractical under realistic rate limits and costs.
Benchmark evasion: detect known benchmark prompts and route them to the genuine model.

Bottom line

Adaptive providers can make software-only audits query-prohibitive without visibly degrading quality.

Adversarial countermeasures: randomized routing + benchmark evasion

Software-Only Auditing Methods Are Fundamentally Unreliable

Three failure modes observed under realistic deployment conditions

Method type	Core weakness	Practical limitation
Text-output statistical tests (e.g., MMD)	Subtle substitutions yield small distribution shifts	Query-prohibitive sample complexity; fragile under temperature and randomized routing
Log-probability fingerprinting	Inference nondeterminism changes logprobs across stacks	Cannot separate infra drift (software/hardware) from substitution; vendor may not expose stable logprobs
Adversarial countermeasures	Provider actively adapts (routing, benchmark evasion)	Audits become gameable; detection becomes exponentially harder as routing probability p ↓

Summary grounded in arxiv 2504.04715 results

Trusted Execution Environments Provide Provable Cryptographic

What TEEs provide

Hardware-secured enclave isolates code + weights.
Remote attestation proves what is running.
Verifies specific weights are loaded for inference.

Why this beats software tests

Provable: cryptographic evidence, not statistics.
Robust: unaffected by sampling noise / temperature.
Actionable: practical today vs ZKPs for large models.

Deployment path

Example: NVIDIA confidential computing on H100 GPUs enables attested inference with modest overhead.
Goal: verifiable inference contracts between API providers and customers.

TEE-backed attestation is the proposed actionable solution

Key Takeaways: Close the Verification Gap with Hardware Attest

Model substitution is economically motivated and hard to observe through a black-box API.

FP8 quantization can preserve benchmark scores (often within error bars).

Production nondeterminism defeats log-probability fingerprinting.

Randomized routing + benchmark evasion make statistical tests query-prohibitive.

Trusted Execution Environments offer provable, deployable model integrity via attestation.

Call to action

Move LLM API integrity from probabilistic tests to verifiable inference with TEEs.

Source: benchmark/source_brief.md